ClassGPT: Plan de Trabajo y Reparto de Tareas

Sprint 2 (14 – 27 Octubre 2026) · MongoDB Atlas, Crawlers, Reranking con Cohere y Comparativa RAG

Proyecto:	ClassGPT — Asistente Docente RAG Inteligente	**Periodo Sprint 2:**	14 al 27 de octubre de 2026 (2 semanas)
Director:	Dr. Pablo Cerro Cañizares	**Próxima Revisión:**	Martes 27 de octubre de 2026, 17:00 h
Titulación:	Grado en Ingeniería del Software (UCM)	**Integrantes:**	Lucas Sánchez · Alberto Zurita · Sergio Barbas

1. Contexto y Decisiones Clave de Dirección (Acta 13 de Octubre)

El presente plan de trabajo operacionaliza los compromisos adquiridos en la sesión técnica de seguimiento del 13 de octubre y las pautas metodológicas transmitidas por la dirección. Tras validar el primer prototipo funcional sobre MongoDB, se acordaron las siguientes directrices prioritarias para las próximas dos semanas:

Acuerdo de Dirección (Acta 13-Oct)	Decisión Técnica y Alcance para el Sprint 2
Foco exclusivo en MongoDB (Acta Sec. 10)	No dispersar el esfuerzo en otras materias (Neo4j/Redis). Consolidar el caso de uso de MongoDB ampliando la extracción a todos los comandos oficiales e incorporar código fuente como material de conocimiento.
Evolución del Almacenamiento (Acta Sec. 05)	**Sustituir el archivo local .pkl por MongoDB Atlas Vector Search**. Al ser la tecnología central de la asignatura, migrar el índice vectorial y metadatos a la nube de MongoDB dota al proyecto de infraestructura nativa y escalable.
Integración de Cohere API (Correo y Acta Sec. 07)	Utilizar la clave de API de Cohere provista por el tutor para incorporar **Reranking semántico** sobre los candidatos recuperados, optimizando la relevancia de los fragmentos antes de enviarlos al modelo de lenguaje.
Estudio de Tipos de RAG (Acta Sec. 02 y 10)	Estudiar y comparar experimentalmente variantes de RAG: búsqueda exclusivamente densa frente a recuperación híbrida (BM25 + embeddings), así como evaluar el efecto de diferentes configuraciones de solapamiento (overlap).
Auto-generación del Benchmark (Acta Sec. 08 y 11)	Automatizar la generación de preguntas de evaluación mediante LLM: construir casos conocidos, alteraciones sintácticas adversariales y errores habituales de alumnos observados en prácticas de cursos anteriores.
Transparencia y Auditoría Web (Acta Sec. 05, 07 y 08)	Renderizar Markdown completo en la web, mostrar el texto exacto del fragmento recuperado (no solo la cita) y habilitar un botón **'Guardar para auditar'** para capturar preguntas y respuestas problemáticas.

2. Objetivos Principales del Sprint 2 (Hito 27 de Octubre de 2026)

- **O1 (Ingesta, IDL y Crawlers):** Catálogo completo de comandos IDL oficiales de MongoDB y crawler/extractor de código desde GitHub.
- **O2 (Persistencia Nativa):** Migración definitiva desde vector_store_mongodb.pkl a MongoDB Atlas Vector Search.
- **O3 (Recuperación y Reranking):** Implementación de Cohere Rerank y comparativa sistemática de variantes de RAG (Denso vs Híbrido BM25).
- **O4 (Frontend y Auditoría):** Interfaz con Markdown enriquecido, visualización de fragmentos de evidencia y botón de captura de fallos.
- **O5 (Evaluación Automatizada):** Generación automática con LLM de preguntas adversariales y benchmark de evaluación ampliado.

3. Reparto Detallado de Tareas Individuales (Sprint 2)

3.1. Lucas Sánchez Martín — Catálogo IDL, Crawlers de GitHub y Migración a MongoDB Atlas

• **Tarea L1 (Prioridad Máxima - Acta Sec. 10): Catálogo Completo de Comandos IDL de MongoDB.**
Extender MongoIDLParser para ingerir y estructurar los comandos nucleares de la asignatura: aggregate_command.idl, createIndexes_command.idl, insert_command.idl, update_command.idl, delete_command.idl y count_command.idl. Extraer esquemas de parámetros y generar ejemplos canónicos contrastados en Shell y PyMongo.
• **Tarea L2 (Prioridad Máxima - Acta Sec. 05): Migración de Persistencia a MongoDB Atlas Vector Search.**
Eliminar la dependencia del fichero local .pkl. Desplegar e integrar un clúster en MongoDB Atlas para alojar los 1.034+ fragmentos y sus embeddings vectoriales, implementando un AtlasVectorStore compatible con la API de búsqueda vectorial nativa.
• **Tarea L3 (Prioridad Alta - Acta Sec. 10 y Sugerencias de Equipo): Crawler y Extractor de Repositorios GitHub.**
Desarrollar un crawler/conector capaz de clonar e inspeccionar repositorios de GitHub (ej. repositorio oficial de MongoDB para descargar los .idl y repositorios de código de la asignatura), extrayendo ficheros de código Python (.py) con preservación sintáctica.
• **Tarea L4 (Prioridad Media - Acta Sec. 06): Pipeline de Ingesta Unificado para Atlas.**
Actualizar mongo_corpus.py para sincronizar directamente los materiales procesados (PDFs, notebooks, IDL y código) en las colecciones de MongoDB Atlas, garantizando indexación idempotente y sin duplicados.

3.2. Alberto Zurita Aguado — Reranking con Cohere, Frontend Markdown y Resiliencia

• **Tarea A1 (Prioridad Máxima - Correo Dirección y Acta Sec. 07): Integración de Cohere Rerank.**
Implementar el módulo CohereReranker en src/retrieval/rerank.py usando la API key oficial de Cohere provista por Pablo. Reordenar los candidatos recuperados antes de la composición del prompt, evaluando la mejora en la posición de la evidencia correcta.
• **Tarea A2 (Prioridad Alta - Acta Sec. 05 y 07): Renderizado Markdown y Visualización de Fragmentos.**
Actualizar templates/index.html para incorporar renderizado completo de Markdown (con formateo de tablas y syntax highlighting en bloques de código) y añadir un componente desplegable para que el alumno pueda leer el texto original del fragmento utilizado como fuente.
• **Tarea A3 (Prioridad Alta - Acta Sec. 08): Botón 'Guardar para Auditar'.**
Añadir en la interfaz un control interactivo que permita registrar consultas con respuestas problemáticas o alucinaciones, persistiendo la traza completa (pregunta, evidencias recuperadas, similitudes y respuesta) en data/auditoria/ para análisis del equipo.
• **Tarea A4 (Prioridad Media - Acta Sec. 04): Resiliencia y Mitigación de Errores de Servidor (500).**
Reforzar el tratamiento de fallos en src/assistant.py para gestionar tiempos de espera prolongados y caídas de API (HTTP 500/503), mostrando estados de diagnóstico claros y evitando interrupciones abruptas durante las demostraciones.

3.3. Sergio Barbas Vázquez — Estudio de Tipos de RAG, Benchmark Auto-Generado y Evaluación

• **Tarea S1 (Prioridad Máxima - Acta Sec. 08 y 11): Auto-generación del Benchmark con LLM.**
Automatizar mediante un script Python la generación sistemática de preguntas y respuestas esperadas utilizando un LLM: preguntas factuales, de sintaxis, comparativas y casos fuera de temario, distinguiendo casos automáticos de casos revisados manualmente.
• **Tarea S2 (Prioridad Alta - Acta Sec. 08 y 11): Casos Adversariales y Errores Habituales de Prácticas.**
Incorporar a la suite de pruebas casos con alteraciones deliberadas (nombres de comandos modificados, parámetros invertidos, contradicciones) y errores frecuentes extraídos de las prácticas de cursos anteriores facilitados por el tutor.
• **Tarea S3 (Prioridad Alta - Acta Sec. 02 y 10): Estudio Comparativo de Variantes de RAG.**
Diseñar un marco experimental para comparar variantes de recuperación sobre el benchmark: (i) RAG Denso simple; (ii) RAG Híbrido (Denso + BM25 léxico); y (iii) RAG con Reranking de Cohere, evaluando asimismo el impacto del solapamiento entre fragmentos (overlap).
• **Tarea S4 (Prioridad Media - Acta Sugerencias): Desacoplamiento de Métricas de Retrieval y Generación.**
Instrumentar el evaluador para medir por separado métricas de recuperación (Recall@k, Precision@k, MRR) de las métricas de calidad docente (fidelidad al material, exactitud de abstención y respeto a no entregar código resuelto).

4. Matriz de Tareas, Entregables y Prioridades del Sprint 2

Tarea / Entregable	Responsable	Módulo Afectado	Prioridad	Criterio de Éxito
Catálogo completo IDL	Lucas	src/parsers/idl_parser.py	**MÁXIMA**	6+ comandos IDL oficiales con ejemplos listos
Migración a MongoDB Atlas	Lucas	src/retrieval/atlas_store.py	**MÁXIMA**	Eliminación de .pkl; búsqueda en Atlas cloud
Crawler GitHub / Código	Lucas	src/ingestion/github_crawler.py	ALTA	Descarga auto de IDL y código de repositorios
Integración Cohere Rerank	Alberto	src/retrieval/rerank.py	**MÁXIMA**	Reordenación semántica con API de Cohere
Frontend Markdown y extractos	Alberto	templates/index.html	ALTA	Markdown completo + vista de fragmentos
Botón 'Guardar para auditar'	Alberto	app.py / assistant.py	ALTA	Persistencia de incidencias en data/auditoria/
Auto-generación de pruebas LLM	Sergio	tests/generate_benchmark.py	**MÁXIMA**	Suite automatizada de 25-30 preguntas
Casos adversariales y errores	Sergio	data/preguntas_adversariales	ALTA	Pruebas con errores reales de alumnos
Comparativa de tipos de RAG	Sergio	research/comparativa_rag.md	ALTA	Estudio: Denso vs Híbrido (BM25) vs Cohere

5. Marco Tecnológico y Decisiones de Arquitectura del Sprint 2

El diseño tecnológico del Sprint 2 traslada a la práctica los compromisos del acta técnica y sienta las bases operativas de la plataforma:

Área de Arquitectura	Tecnología / Enfoque	Justificación y Beneficio para el TFG
Persistencia Vectorial (Lucas)	MongoDB Atlas Vector Search (sustituye a .pkl)	Almacenamiento nativo en la base de datos de la asignatura. Permite búsquedas vectoriales indexadas con filtros escalables en cloud, desacoplando la memoria local del servidor.
Adquisición y Crawling (Lucas)	Git Crawler + PyYAML para GitHub oficial	Automatiza la captura de comandos .idl actualizados desde el repositorio de MongoDB y la extracción de código de proyectos de prácticas, evitando cargas manuales estáticas.
Reranking Semántico (Alberto)	Cohere Rerank API (rerank-v3.5)	Reordena el conjunto recuperado mediante un cross-encoder de última generación facilitado por la dirección, elevando la precisión del contexto antes de enviarlo a Gemini.
Experimentación RAG (Sergio)	Dense (MiniLM) vs BM25 + Evaluación de Overlap	Estudio empírico riguroso para responder a la petición del acta: determinar cuantitativamente qué combinación de RAG ofrece mayor Recall@k y qué porcentaje de solapamiento es óptimo.

Nota sobre la Planificación de GraphRAG: En estricta concordancia con el acta del 13 de octubre y el correo del director, la construcción del Knowledge Graph se mantiene explícitamente en pausa. Todo el esfuerzo de este ciclo se focaliza en consolidar la base de conocimiento en Atlas, el reranking y la experimentación comparativa.

6. Criterios de Aceptación para la Reunión del 27 de Octubre de 2026

• **1. Demostración en vivo sobre MongoDB Atlas:** Consultas ejecutadas en tiempo real recuperando fragmentos vectoriales directamente desde MongoDB Atlas Vector Search, habiendo prescindido de los ficheros locales .pkl.

• **2. Extracción Ampliada de Comandos IDL:** Cobertura demostrada para aggregate, creación de índices y operaciones CRUD con sus ejemplos canónicos enriquecidos.

• **3. Reranking con Cohere Operativo:** Demostración del flujo con Cohere activo en la web, contrastando el orden de fragmentos con y sin reranking.

• **4. Interfaz con Markdown y Panel de Auditoría:** Web con visualización de texto de evidencias y captura funcional de incidencias mediante el botón de auditoría.

• **5. Informe Comparativo de Variantes de RAG:** Entrega del informe con la comparativa empírica entre RAG Denso, Híbrido (BM25) y Reranking, evaluado sobre el benchmark auto-generado con casos adversariales.

Lucas Sánchez Martín
Ingeniería de Ingesta, IDL & Atlas

Alberto Zurita Aguado
Reranking, Web & Resiliencia

Sergio Barbas Vázquez
Benchmark, QA & Comparativa RAG